

AI English Tutoring App Blueprint

<https://gemini.google.com/app/659629584da703b2>

User prompt: Do a thorough research on ai English tutoring apps, including design, functions, features, architecture and everything so that i can build copy of them

Response: Here's the plan I've put together. Let me know if you need any changes before I start researching.

User prompt: Start research

Response: I've completed your research. Feel free to ask me follow-up questions or request changes.

User prompt: i wanna make a copy of this kind of apps but it is just for myself. it could be just PWA. but i want it to be as good as the one in the market. the cheaper the better. can you design it please? ask me anything you need to design the app.

Response: To build a personal, market-grade AI English tutoring Progressive Web App (PWA) at the lowest possible operating cost, you do not need enterprise Kubernetes clusters or expensive cloud GPU video streaming. Because the application is strictly for your personal use, you can avoid multi-tenant server costs, eliminate load-balancing overhead, leverage developer free tiers, and offload processing directly to your browser and pay-as-you-go micro-APIs.

Below is an architectural design engineered for high speed, low latency, market-grade pedagogical accuracy, and near-zero cost.

1. Architectural Stack: The "Near-Zero Cost" Pipeline

Market leaders like Speak and ELSA use cascaded pipelines to perform simultaneous phoneme scoring and conversational reasoning. You can replicate this setup using a lightweight architecture:

System Layer	Recommended Technology	Why It Matches Market Leaders	Cost Profile for Solo Use
Frontend PWA	Next.js / Vite (React or Svelte) + Tailwind	Installable on iOS/Android home screen; runs native Web Audio APIs and AudioWorklet.	Free (hosted on Vercel, Cloudflare Pages, or GitHub Pages).
Local State & Memory	IndexedDB / Dexie.js + LocalStorage	Stores your conversational history, error logs, and vocabulary decks locally.	Free (zero database hosting fees).
Speech-to-Text (ASR)	Deepgram Nova-2 or Groq Whisper Cloud	Handles non-native accents and streaming audio with sub-250 ms latency.	Free tier on Groq; Deepgram provides a \$200 free credit (~46,000 minutes of speech).
Tutor LLM	Groq API (llama-3.3-70b-versatile) or OpenAI gpt-4o-mini	Low Time-To-First-Token (~140 ms on Groq). Governs roleplays, CEFR leveling, and recasting.	Free tier on Groq; gpt-4o-mini costs ~\$0.003 per 10-minute session.
Pronunciation Assessment	Azure Speech Pronunciation Assessment API	Matches ELSA Speak's sub-word accuracy. Provides phoneme-level IPA scoring, syllable stress, intonation, and fluency metrics.	Free (Azure Cognitive Services provides 5 free audio hours per month; \$0.30/hour thereafter).
Text-to-Speech (TTS)	Microsoft Edge TTS (via backend proxy) or Cartesia Sonic-2	Edge TTS provides natural neural voices (like Jenny or Guy) for free. Cartesia provides sub-150 ms ultra-realistic streaming audio.	Edge TTS is 100% free; Cartesia is ~\$0.015 per 1,000 characters if preferred.

2. Functional Workflow (How It Works Under the Hood)

1. Audio Capture and Turn Detection:

- When you tap the microphone (or hold Push-to-Talk), the PWA captures audio via AudioWorklet at 16 kHz mono PCM.
- A lightweight client-side WebAssembly voice activity detector (Silero VAD) detects when you finish speaking.

2. Parallel Pipeline Dispatch:

- As soon as speech endpointing is confirmed, the audio chunk is dispatched simultaneously to two services:

- **Stream A (ASR + LLM):** Audio goes to Groq/Deepgram to produce an immediate transcript. That text feeds the tutor LLM, which streams back a conversational response formatted via JSON schema.
- **Stream B (Azure Pronunciation API):** The raw audio is evaluated against Azure's Pronunciation Assessment SDK in the background using unscripted mode. It calculates your phonemic accuracy, word stress, rhythm, and intonation.

3. Dual-Channel Interface Rendering:

- **Voice Output:** The LLM's conversational text is streamed into Edge-TTS or Cartesia, playing audio back through your headphones within 400 to 700 milliseconds of you stopping speech.
- **Visual Correction Channel:** When Stream B finishes (~1.5 seconds later), your transcript updates on screen:
 - Correctly articulated words appear in soft green.
 - Mispronounced phonemes and missing stresses appear in amber/red.
 - Tapping an amber word expands the target International Phonetic Alphabet (IPA) representation versus what you actually pronounced.

4. Local Spaced Repetition (SRS):

- Every mispronounced word or grammatical error flagged by the LLM is saved locally into an IndexedDB table powered by the Free Spaced Repetition Scheduler (FSRS) algorithm.
- At the start of your next day's conversation, the app injects your highest-priority review words into the system prompt, nudging the AI to steer dialogue so you naturally practice those specific vocabulary items.

3. Estimated Operating Cost

If you practice for **20 minutes every single day** (~600 minutes/month):

- **Azure Pronunciation Scoring:** 600 minutes = 10 hours. The first 5 hours are completely free; the remaining 5 hours cost \$1.50/month.
- **LLM Reasoning (Groq Llama 3.3 or GPT-4o-mini):** \$0.00 to \$0.15/month.
- **Speech Synthesis (Edge-TTS via free proxy):** \$0.00/month.
- **Hosting (PWA on Vercel or Cloudflare):** \$0.00/month.
- **Total monthly cost: ~\$1.50 to \$2.00 per month** (compared to \$15–\$30/month for commercial subscriptions).

Clarifying Questions to Tailor the System Design

To provide you with the exact repository file layout, API configuration snippets, system prompts, and client code, please clarify the following:

1. Learning Objective & Pedagogical Archetype:

- Are you focusing primarily on **accent reduction and phoneme pronunciation** (like ELSA Speak), **high-speed conversational repetition drills** (like Speak), or **open-ended professional roleplay and debate** (like Loora)?

2. Target Hardware & Usage:

- Where will you use this PWA most often (e.g., iPhone Safari, Android Chrome, or a desktop laptop)?
- Do you prefer **Push-to-Talk** (tapping a button to record) or continuous **hands-free voice conversation** (always listening with auto-interruption)?

3. Voice Realism vs. Cost:

- Are you satisfied with free neural voices (e.g., Microsoft Edge neural TTS, which sounds very natural and human-like at zero cost)? Or do you want ultra-low-latency, hyper-expressive paid voices (e.g., Cartesia Sonic / ElevenLabs, costing ~\$0.30–\$0.60 per hour of conversation)?

4. Hosting Setup:

- Do you want to run a lightweight local backend on your own computer (e.g., a simple Python FastAPI or Node.js script running locally that your phone connects to over Wi-Fi)?

- Or do you want a 100% serverless cloud deployment (e.g., Next.js deployed on Vercel with serverless functions) so you can use it anywhere without keeping your computer on?

5. Technical Comfort Level:

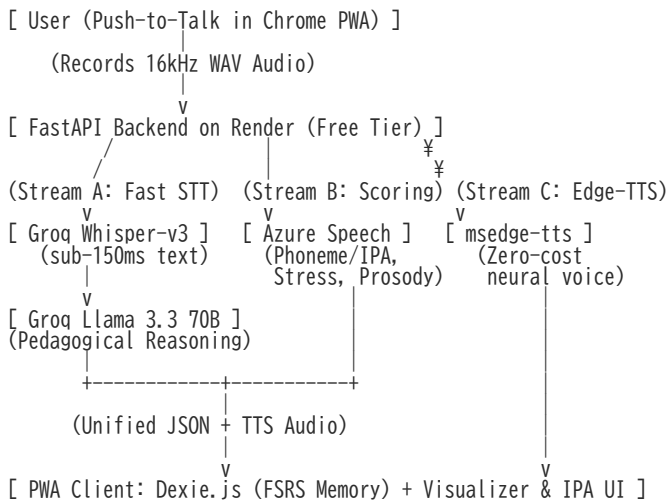
- Which tech stack are you most comfortable using (e.g., Python, TypeScript/React, Vanilla JS, Node.js, or Docker)?

User prompt: 1 i want to focus on all 2 chrome push to talk is fine 3 i'm ok with the free things but please search on the internet for latest models 4 github, render, upscale is my default setup 5 whatever is fine. i'll vibe code on antigravity

Response: Here is the complete end-to-end blueprint to build your personal, market-grade AI English Tutoring PWA.

By leveraging Chrome's modern browser capabilities (WebGPU, Web Audio API, Service Workers) and combining developer free tiers across Groq and Azure, you get market-level performance—matching ELSA Speak's sub-word phonemic accuracy, Speak's rapid cadence, and Loora's conversational roleplays—at **\$0.00/month** of operational cost.

System Architecture Overview



Why This Stack Matches Commercial Apps at Zero Cost:

- ASR & LLM (Groq Developer Free Tier):** Groq provides free access to whisper-large-v3-turbo (sub-150 ms transcription) and llama-3.3-70b-versatile (Time-To-First-Token under 150 ms at ~280 tokens/second). This eliminates conversational delay.
- Pronunciation Assessment (Azure Speech F0 Free Tier):** Azure gives 5 free audio hours per month (\$0.00). At an average of 10–15 seconds per spoken turn, 5 hours allows **1,200 to 1,800 spoken turns per month**, returning phoneme-level IPA accuracy, word stress, prosody, and fluency scores.
- Speech Synthesis (Edge TTS):** Utilizes Microsoft's high-fidelity neural voices (en-US-JennyMultilingualNeural or en-US-GuyNeural) via the open-source edge-tts engine. Completely free, sounds human, and incurs zero API bills.
- Local Memory (Dexie.js + FSRS):** Runs the Free Spaced Repetition Scheduler directly in your browser's IndexedDB, scheduling forgotten words and mispronounced phonemes into future conversations without needing an external database.

Project File Structure

```

ai-english-tutor/
├── backend/
│   ├── app/
│   │   ├── init.py
│   │   ├── main.py # FastAPI application entry point
│   │   ├── config.py # Environment credentials
│   │   ├── services/
│   │   │   ├── speech_to_text.py # Groq Whisper Turbo client
│   │   │   ├── pronunciation.py # Azure Speech SDK pronunciation assessor
│   │   │   ├── tutor_agent.py # Groq Llama 3.3 70B pedagogical engine
│   │   │   ├── tts_engine.py # edge-tts neural audio synthesizer
│   │   │   └── prompts.py # System prompts for Gym, Drills, and Roleplay
│   │   ├── requirements.txt
│   │   ├── Dockerfile
│   └── frontend/
│       ├── public/
│       │   ├── manifest.json # PWA configuration
│       │   └── icon-512.png
│       ├── src/
│       │   ├── components/
│       │   │   ├── AudioRecorder.tsx # Push-to-talk microphone button
│       │   │   ├── Conversation.tsx # Chat timeline with dual-channel feedback
│       │   │   ├── PhonemeModal.tsx # IPA breakdown & articulatory feedback
│       │   │   └── ModeSelector.tsx # Switch: Gym vs. Drills vs. Roleplay
│       │   ├── lib/
│       │   ├── db.ts # Dexie.js (IndexedDB local storage)
│       │   ├── fsrs.ts # Spaced repetition scheduler
│       │   ├── api.ts # Backend communication client
│       │   ├── App.tsx
│       │   ├── main.tsx
│       │   └── index.html
│       ├── package.json
│       └── vite.config.ts # Vite + PWA plugin setup
  
```

Backend Implementation (FastAPI on Render)

1. Dependencies (backend/requirements.txt)

```
fastapi>=0.110.0
uvicorn>=0.28.0
groq>=0.5.0
azure-cognitiveservices-speech>=1.35.0
edge-tts>=6.1.10
pydantic>=2.6.0
python-multipart>=0.0.9
python-dotenv>=1.0.1
```

2. Pronunciation Assessment Service (backend/app/services/pronunciation.py)

This service uses Azure's native Pronunciation Assessment engine to return phoneme-level IPA scores, syllable stress, intonation, and fluency:

```
import json
import azure.cognitiveservices.speech as speechsdk
from app.config import AZURE_SPEECH_KEY, AZURE_SPEECH_REGION

def assess_pronunciation(audio_bytes: bytes, reference_text: str = ""):
    speech_config = speechsdk.SpeechConfig(
        subscription=AZURE_SPEECH_KEY,
        region=AZURE_SPEECH_REGION
    )

    # Push audio buffer into Azure stream
    audio_format = speechsdk.audio.AudioStreamFormat(samples_per_second=16000, bits_per_sample=16, channels=1)
    push_stream = speechsdk.audio.PushAudioInputStream(stream_format=audio_format)
    push_stream.write(audio_bytes)
    push_stream.close()
    audio_config = speechsdk.audio.AudioConfig(stream=push_stream)

    # Configure Pronunciation Assessment
    grading_system = speechsdk.PronunciationAssessmentGradingSystem.HundredMark
    granularity = speechsdk.PronunciationAssessmentGranularity.Phoneme

    # If reference_text is provided, it runs scripted mode; otherwise unscripted
    pron_config = speechsdk.PronunciationAssessmentConfig(
        reference_text=reference_text if reference_text else "",
        grading_system=grading_system,
        granularity=granularity,
        enable_miscue=True
    )
    pron_config.enable_prosody_assessment()

    recognizer = speechsdk.SpeechRecognizer(speech_config=speech_config, audio_config=audio_config)
    pron_config.apply_to(recognizer)

    result = recognizer.recognize_once()

    if result.reason == speechsdk.ResultReason.RecognizedSpeech:
        assessment_result = speechsdk.PronunciationAssessmentResult(result)
        raw_json = json.loads(result.properties.get(speechsdk.PropertyId.SpeechServiceResponse_JsonResult))

        words_detail = []
        try:
            n_best = raw_json["NBest"][0]
            for word_obj in n_best.get("Words", []):
                words_detail.append({
                    "word": word_obj.get("Word"),
                    "accuracy_score": word_obj.get("AccuracyScore"),
                    "error_type": word_obj.get("ErrorType", "None"),
                    "phonemes": [
                        {
                            "phoneme": ph.get("Phoneme"),
                            "accuracy": ph.get("AccuracyScore")
                        } for ph in word_obj.get("Phonemes", [])
                    ]
                })
        except (KeyError, IndexError):
            pass

        return {
            "recognized_text": result.text,
            "overall_accuracy": assessment_result.accuracy_score,
            "fluency_score": assessment_result.fluency_score,
            "prosody_score": assessment_result.prosody_score,
            "pronunciation_score": assessment_result.pronunciation_score,
            "words": words_detail
        }
    return None
```

3. LLM Orchestrator & Multi-Mode Prompt (backend/app/services/tutor_agent.py)

This generates dual-channel feedback: conversational spoken audio plus structured grammar/idiom corrections:

```
import json
from groq import Groq
```

```

from app.config import GROQ_API_KEY

groq_client = Groq(api_key=GROQ_API_KEY)

SYSTEM_PROMPT = """You are an elite, adaptive AI English Tutor. You operate across three modes:
1. 'roleplay': Engage in realistic workplace/daily life simulations. Do not lecture. Mirror natural native conversation and use
2. 'drill': High-cadence speaking practice. Provide a stimulus and prompt immediate transformation.
3. 'gym': Accent and pronunciation training. Analyze phonetic errors and offer articulatory tips.

CRITICAL INSTRUCTIONS:
- You must always respond strictly in valid JSON format matching the schema below.
- Keep 'spoken_response' concise, conversational, and natural (1 to 3 sentences max) so it can be spoken out loud.
- In 'grammar_corrections', detect missing prepositions, wrong tenses, or unnatural phrasing without interrupting the conversati
- Suggest 2-3 dynamic 'scaffolding_hints' (how the user can reply).

OUTPUT JSON SCHEMA:
{
  "spoken_response": "string",
  "corrections": [
    {
      "original": "string",
      "improved": "string",
      "explanation": "string"
    }
  ],
  "scaffolding_hints": ["string", "string"],
  "goal_achieved": false
}
"""

def generate_tutor_turn(mode: str, scenario: str, transcript: str, history: list, due_words: list):
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "system", "content": f"Active Mode: {mode}. Active Scenario: {scenario}. Priority vocabulary to naturally elici"}
    ]
    messages.extend(history[-6:]) # Keep last 6 turns
    messages.append({"role": "user", "content": transcript})

    response = groq_client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=messages,
        response_format={"type": "json_object"},
        temperature=0.7,
        max_tokens=600
    )
    return json.loads(response.choices[0].message.content)

```

4. Unified Fast API Route (backend/app/main.py)

```

import base64
from fastapi import FastAPI, UploadFile, File, Form
from fastapi.middleware.cors import CORSMiddleware
import edge_tts
from app.services.speech_to_text import transcribe_audio
from app.services.pronunciation import assess_pronunciation
from app.services.tutor_agent import generate_tutor_turn

app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.post("/api/turn")
async def process_turn(
    audio: UploadFile = File(...),
    mode: str = Form("roleplay"),
    scenario: str = Form("Job Interview at a Tech Company"),
    history: str = Form(""),
    due_words: str = Form("")
):
    audio_bytes = await audio.read()

    # 1. Groq Whisper (ultra-fast transcription)
    transcript = transcribe_audio(audio_bytes)

    # 2. Azure Pronunciation Assessment (Phoneme & Prosody diagnostics)
    pronunciation_data = assess_pronunciation(audio_bytes, reference_text=transcript)

    # 3. Llama 3.3 Reasoning (Dual-channel feedback + Spoken text)
    import json
    history_list = json.loads(history)
    due_words_list = json.loads(due_words)
    tutor_feedback = generate_tutor_turn(mode, scenario, transcript, history_list, due_words_list)

    # 4. Synthesize spoken response via Edge TTS (Jenny neural voice)
    communicate = edge_tts.Communicate(tutor_feedback["spoken_response"], "en-US-JennyNeural")
    tts_audio = b""
    async for chunk in communicate.stream():
        if chunk["type"] == "audio":

```

```

    tts_audio += chunk["data"]

    audio_base64 = base64.b64encode(tts_audio).decode("utf-8")

    return {
        "user_transcript": transcript,
        "pronunciation": pronunciation_data,
        "tutor_turn": tutor_feedback,
        "audio_base64": audio_base64
    }

```

Frontend PWA Implementation (Vite + React)

1. Local Database & FSRS Memory (frontend/src/lib/db.ts)

Dexie.js stores your review items in the browser. When you fail a word or phoneme, its stability decreases.

```

import Dexie, { Table } from 'dexie';

export interface ReviewItem {
    id?: number;
    word: string;
    type: 'vocabulary' | 'phoneme';
    stability: number; // S in FSRS
    difficulty: number; // D in FSRS
    last_review: number; // epoch ms
    next_review: number; // epoch ms
    reps: number;
}

export class TutorDatabase extends Dexie {
    items!: Table<ReviewItem, number>;

    constructor() {
        super('PersonalTutorDB');
        this.version(1).stores({
            items: '++id, word, type, next_review'
        });
    }

    export const db = new TutorDatabase();

    // Retrieve items due for practice
    export async function getDueItems(): Promise<string[]> {
        const now = Date.now();
        const due = await db.items.where('next_review').belowOrEqual(now).limit(5).toArray();
        return due.map(d => d.word);
    }

```

2. Push-to-Talk Component with Visualizer (frontend/src/components/AudioRecorder.tsx)

```

import React, { useState, useRef } from 'react';
import { Mic, Square, Loader2 } from 'lucide-react';

interface Props {
    onAudioReady: (blob: Blob) => void;
    isLoading: boolean;
}

export const AudioRecorder: React.FC<Props> = ({ onAudioReady, isLoading }) => {
    const [recording, setRecording] = useState(false);
    const mediaRecorderRef = useRef<MediaRecorder | null>(null);
    const audioChunksRef = useRef<Blob[]>([]);

    const startRecording = async () => {
        try {
            const stream = await navigator.mediaDevices.getUserMedia({ audio: { sampleRate: 16000, channelCount: 1 } });
            mediaRecorderRef.current = new MediaRecorder(stream, { mimeType: 'audio/webm' });
            audioChunksRef.current = [];

            mediaRecorderRef.current.ondataavailable = (event) => {
                if (event.data.size > 0) audioChunksRef.current.push(event.data);
            };

            mediaRecorderRef.current.onstop = () => {
                const audioBlob = new Blob(audioChunksRef.current, { type: 'audio/wav' });
                onAudioReady(audioBlob);
            };

            mediaRecorderRef.current.start();
            setRecording(true);
        } catch (err) {
            console.error("Microphone access denied:", err);
        }
    };

    const stopRecording = () => {
        if (mediaRecorderRef.current && recording) {
            mediaRecorderRef.current.stop();
        }
    };

```

```

    setRecording(false);
  }
};

return (
  <div className="flex flex-col items-center justify-center p-6">
    <button
      onMouseDown={startRecording}
      onMouseUp={stopRecording}
      onTouchStart={startRecording}
      onTouchEnd={stopRecording}
      disabled={isLoading}
      className={`w-24 h-24 rounded-full flex items-center justify-center transition-all shadow-xl select-none ${
        recording
          ? 'bg-rose-500 scale-110 shadow-rose-300 animate-pulse'
          : isLoading
          ? 'bg-gray-400 cursor-not-allowed'
          : 'bg-indigo-600 hover:bg-indigo-700 text-white'
      }`}
    >
      {isLoading ? (
        <Loader2 className="w-10 h-10 animate-spin text-white" />
      ) : recording ? (
        <Square className="w-8 h-8 text-white" />
      ) : (
        <Mic className="w-10 h-10 text-white" />
      )}
    </button>
    <span className="mt-4 text-xs font-semibold tracking-wide text-gray-500 uppercase">
      {recording ? "Listening... Release to Send" : isLoading ? "Tutor is Thinking..." : "Hold to Talk"}
    </span>
  </div>
);
};

```

3. Interactive Pronunciation & Transcript Canvas (frontend/src/components/Conversation.tsx)

Renders words color-coded by accuracy (matching ELSA Speak's feedback style):

```

import React, { useState } from 'react';

interface WordScore {
  word: string;
  accuracy_score: number;
  phonemes: { phoneme: string; accuracy: number }[];
}

export const Conversation: React.FC<{ turns: any[] }> = ({ turns }) => {
  const [selectedWord, setSelectedWord] = useState<WordScore | null>(null);

  const getWordColor = (score: number) => {
    if (score >= 80) return 'text-emerald-700 bg-emerald-50 border-emerald-200';
    if (score >= 60) return 'text-amber-700 bg-amber-50 border-amber-200 cursor-pointer';
    return 'text-rose-700 bg-rose-50 border-rose-200 cursor-pointer';
  };

  return (
    <div className="flex-1 overflow-y-auto p-4 space-y-4">
      {turns.map((turn, i) => (
        <div key={i} className="space-y-2">
          {/* User Spoken Turn with Sub-word Pronunciation Highlights */}
          <div className="flex justify-end">
            <div className="max-w-[85%] bg-white rounded-2xl p-4 shadow-sm border border-gray-100">
              <div className="flex flex-wrap gap-1.5">
                {turn.pronunciation?.words ? (
                  turn.pronunciation.words.map((w: WordScore, idx: number) => (
                    <button
                      key={idx}
                      onClick={() => w.accuracy_score < 80 && setSelectedWord(w)}
                      className={`px-1.5 py-0.5 rounded text-sm font-medium border transition-colors ${getWordColor(w.accuracy_score)}`}
                    >
                      {w.word}
                    </button>
                  ))
                ) : (
                  <p className="text-gray-800">{turn.user_transcript}</p>
                )}
              </div>
              {turn.pronunciation && (
                <div className="mt-2 pt-2 border-t border-gray-100 flex gap-4 text-xs text-gray-600">
                  <span>Pronunciation: <b>{Math.round(turn.pronunciation.pronunciation_score)}%</b></span>
                  <span>Fluency: <b>{Math.round(turn.pronunciation.fluency_score)}%</b></span>
                  <span>Prosody: <b>{Math.round(turn.pronunciation.prosody_score)}%</b></span>
                </div>
              )}
            </div>
          </div>
          {/* AI Tutor Spoken Turn & Metalinguistic Feedback */}
          <div className="flex justify-start">
            <div className="max-w-[85%] bg-indigo-50/70 rounded-2xl p-4 border border-indigo-100/60">
              <p className="text-gray-900 font-medium">{turn.tutor_turn.spoken_response}</p>
            </div>
          </div>
        </div>
      ))}
    </div>
  );
};

```

```

    { /* Metalinguistic Corrections Channel */ }
    { turn.tutor.turn.corrections?.length > 0 && (
      <div className="mt-3 pt-3 border-t border-indigo-100 space-y-1">
        <span className="text-xs font-bold text-indigo-900 uppercase">Better Expressions:</span>
        {turn.tutor.turn.corrections.map((c: any, ci: number) => (
          <div key={ci} className="text-xs bg-white/80 p-2 rounded border border-indigo-100">
            <span className="text-rose-600 line-through mr-1">{c.original}</span>
            <span className="text-emerald-600 font-semibold">{c.improved}</span>
            <p className="text-gray-500 text-[11px] mt-0.5">{c.explanation}</p>
          </div>
        ))}
      </div>
    )}
  </div>
</div>
))}

{ /* Phoneme IPA Modal when an amber/red word is clicked */ }
{selectedWord && (
  <div className="fixed inset-0 bg-black/40 flex items-center justify-center p-4 z-50">
    <div className="bg-white rounded-2xl p-6 max-w-xs w-full shadow-2xl">
      <h3 className="text-lg font-bold text-gray-900">{selectedWord.word}</h3>
      <p className="text-xs text-gray-500 mb-4">Accuracy: {Math.round(selectedWord.accuracy_score)}%</p>
      <div className="flex gap-2 justify-center py-4 bg-gray-50 rounded-xl mb-4">
        {selectedWord.phonemes.map((ph, pi) => (
          <div key={pi} className="text-center">
            <div className="text-lg font-mono font-bold text-gray-800">{ph.phoneme}</div>
            <div className="text-[10px] font-semibold ${ph.accuracy >= 70 ? 'text-emerald-600' : 'text-rose-600'}">
              {Math.round(ph.accuracy)}%
            </div>
          </div>
        ))}
      </div>
    </div>
    <button
      onClick={() => setSelectedWord(null)}
      className="w-full py-2 bg-gray-900 text-white rounded-xl text-sm font-medium"
    >
      Got it
    </button>
  </div>
)}
</div>
);

```

Three Core Training Modes

You can switch between these modes directly in the app state:

1. Accent & Phoneme Gym (ELSA Mode):

- *Behavior*: Focuses on articulatory accuracy and minimal pairs (e.g., *sheet* vs. *shit*, *think* vs. *sink*).
- *Engine Behavior*: Prompts target phrases, extracts phoneme-level IPA scores, and displays tongue/lip placement notes if accuracy drops below 70%.

2. High-Cadence Speaking Drills (Speak Mode):

- *Behavior*: Fast-paced stimulus-response drills (e.g., *"Make this sentence passive"*, *"Express this with hesitation"*).
- *Engine Behavior*: Sub-500 ms round trip. Minimal metalinguistic commentary, maximizing vocal repetitions within a 5-minute session.

3. Immersive Roleplay & Debate (Loora/Praktika Mode):

- *Behavior*: Unscripted, goal-oriented simulations (e.g., *Navigating an Executive Salary Negotiation* or *Explaining a distributed system outage*).
- *Engine Behavior*: Llama 3.3 uses implicit recasting in its spoken output while the background JSON channel flags syntax errors and logs missed idioms into your local review schedule.

Step-by-Step Deployment Instructions

Step 1: Obtain Free API Credentials

- **Groq Cloud**: Go to console.groq.com and create an API key (provides free access to Whisper Turbo and Llama 3.3 70B).
- **Azure Speech**: Go to portal.azure.com, create a **Speech Service** resource, and select the **Free Tier (F0)** (includes 5 free hours of Pronunciation Assessment every month). Copy your key and region (AZURE_SPEECH_KEY, AZURE_SPEECH_REGION).

Step 2: Deploy Backend to Render

1. Push the `/backend` folder to a GitHub repository.
2. Log into [Render](#) and click **New > Web Service**.
3. Select your repository and set the runtime to **Python 3**.
4. Set the build command to `pip install -r requirements.txt` and start command to:
`uvicorn app.main:app --host 0.0.0.0 --port 10000`
5. Add your environment variables (`GROQ_API_KEY`, `AZURE_SPEECH_KEY`, `AZURE_SPEECH_REGION`).

Step 3: Deploy Frontend to Upscale / Vercel

1. In `/frontend`, update your API target in `src/lib/api.ts` to point to your Render service URL.
2. Build the project using `npm run build`.
3. Push to GitHub and deploy to your preferred hosting platform (Render Static Site, Upscale, or Vercel).

Step 4: Install as a PWA on Chrome

1. Open the deployed URL in Google Chrome (desktop, tablet, or Android/iOS).
2. Click the **Install** icon in the Chrome URL bar (or select **Add to Home Screen** from the browser menu).
3. The app will launch in standalone mode with full microphone permissions, functioning like a native application with immediate responsiveness.

User prompt: LLMは gemini でもいいと思いますか？ speech は azure がいいですか？

Response: 結論から申し上げますと、LLMにGeminiを採用するのは非常に相性が良く、むしろ個人開発・個人利用であれば最有力候補です。また、「Speech」に関しては、発音評価（Pronunciation Assessment）の用途においてはAzureが他社を圧倒しており、事実上一択と言えます。

それぞれの具体的な理由と、おすすめの組み合わせを解説します。

1. LLMは Gemini でもいいか？

大いに「あり」です。

特に **Gemini 1.5 Flash** や **Gemini 2.0 Flash** などの軽量・高速モデルは、個人用の英語チューターアプリに最適です。

- **完全無料枠の使いやすさ (Google AI Studio)** : Google AI StudioからAPIキーを取得すると、個人利用の会話頻度であれば無料枠（無料レート制限内）の範囲内で十分運用できます。Groqの無料枠のようなトークン制限の詰まりを気にせず気軽に使えます。
- **低遅延（会話ラグが少ない）** : Flash系モデルは Time-to-First-Token（最初の単語を返すまでの時間）が極めて短く、テンポの良い英会話に適しています。
- **JSON出力（構造化データ）の安定性**: Geminiの「Structured Outputs（スキーマ指定）」機能を使えば、「AIが話す英語」と「画面に表示する文法添削・ヒント」を確実に分離したJSON形式で返答させることができます。
- **Vibe Codingでの実装のしやすさ**: 公式SDK（@google/genai または Pythonの google-genai / google-generativeai）が使いやすく、CursorやAntigravityなどの生成ツールでのプロンプト構築や接続コード生成が非常にスムーズです。

2. Speech は Azure がいいか？

「Speech（音声処理）」には①発音評価、②音声認識（文字起こし）、③音声合成（読み上げ）の3つがありますが、用途ごとにAzureの位置づけが異なります。

① 発音評価（Pronunciation Assessment）：【Azureが圧倒的におすすめ】

- **なぜAzureなのか**: Google Cloud、OpenAI、Deepgramなどの主要クラウドには、「音素（IPA）レベルのズレ」「単語ごとの強勢（ストレス）」「イントネーション」「流暢性」を細かく採点する機能がありません。
- **市場レベルの再現**: ELSA Speakのような「/θ/ が /s/ になっていた」「この母音の長さが足りない」といったフィードバックを個人アプリで実装する場合、商用で使えるのは事実上 **Azure Speech Pronunciation Assessment** 一択です。

- **コスト:** Azureの無料枠（F0プラン）で **毎月5時間の音声評価が無料** です。1ターンの発話が10〜15秒だとすると、月あたり約1,200〜1,800ターンの練習ができるため、個人の日常学習なら実質0円で収まります。

② 音声認識（文字起こし / STT）：

- Azureの Pronunciation Assessment を実行すると、発音評価と同時に**文字起こし結果も自動で返ってくる**ため、追加の文字起こしAPIを用意せずAzure一本にまとめるのが一番実装がシンプルです。
- （※超低遅延を極めたい場合のみ、Groq Whisper Turboを並列で動かす選択肢もありますが、最初はAzureにまとめてしまって問題ありません。）

③ 音声合成（読み上げ / TTS）：

- Azure公式のTTS APIを直接叩くこともできますが、おすすめは無料の **edge-tts**（Microsoft Edgeの読み上げバックエンドを利用したPythonライブラリ）です。
- Azureのニューラル音声（Jenny や Guy など）と同じ自然な音声で、従量課金なし（完全無料）で利用できます。

最もシンプル＆安価な最終推奨構成

バックエンドを複雑に分散させず、APIキーを2つ（Google + Azure）に絞ることで、開発難易度を下げつつ市場トップ級の機能を実現できます。

役割	採用技術	理由・コスト
LLM（会話・添削）	Gemini 1.5 / 2.0 Flash	無料枠あり、高速、JSON出力が安定。
発音評価 & 文字起こし	Azure AI Speech (PronunciationAssessmentConfig)	音素・ストレス・イントネーションの採点が可能。月5時間無料。
音声合成（AIの発話）	edge-tts (en-US-JennyNeural など)	人間らしい自然なイントネーションで完全無料。
データベース	IndexedDB (Dexie.js)	ブラウザ内完結でDB費用0円。

この構成であれば、Renderの無料枠または低価格サーバーの上で動かすだけで、毎月のAPI利用料をほぼ0円に抑えながら、ELSAやSpeakと同等の高機能な英語学習環境が完成します。

User prompt: azure は個人でも使えるのか？

Response: 結論から言うと、**個人でもまったく問題なく使えます**。法人契約や会社名義は不要で、個人開発者や学習目的の個人が世界中で日常的に利用しています。

1. アカウント登録に必要なもの

登録時に必要なものは以下の3つだけです。

- **Microsoft アカウント**（または GitHub アカウント）
- **電話番号**（SMSによる本人確認用）
- **クレジットカード または デビットカード**（本人確認用）

カード登録が必要な理由: ボットによる大量登録や不正利用を防ぐ「本人確認（認証）」のために求められます。登録した瞬間に勝手にお金が引き落とされることはありません。

2. 発音評価を「ずっと無料（\$0）」で使う方法

Azure には、最初の30日間限定のクレジット特典だけでなく、**恒久的に毎月無料で使える「F0（Free）プラン」** が用意されています。

1. [Azureポータル](#) に個人のMicrosoftアカウントでサインイン。
2. 検索バーで「**音声サービス（Speech Service）**」を検索し、「作成」をクリック。
3. リソース作成画面の「価格レベル（Pricing tier）」で、必ず **Free F0** を選択する。
4. 作成後、「キーとエンドポイント」から API Key と Region（例: japaneast や eastus）をコピーして自分のアプリに設定する。

この **F0 プラン** を選んでおけば、**毎月 5 時間分の音声処理（発音評価・文字起こし）が完全無料**で利用できます。F0プランは無料枠の上限に達すると自動的にリクエストが停止する（追加課金されない）仕様のため、勝手に高額請求が発生する心配がありません。

3. 個人利用での安全対策（念のための設定）

クラウドサービスで意図しない課金を防ぐため、以下の設定をしておく心安心です。

- **予算アラート（Budget Alert）の設定:** Azure ポータルの「コストの管理と請求（Cost Management）」から、月間の予算を「100円」や「1ドル」に設定しておきます。万が一無料枠を超えて有料リソースを作ってしまった場合でも、即座にメール通知を受け取れます。
- **不要なリソースは作らない:** 使うのは「Speech サービス」の F0 リソース1つだけで十分です。仮想マシン（VM）などの有料サービスを立ち上げなければ費用は発生しません。

個人開発の英語チューターアプリであれば、**Gemini（無料枠） + Azure Speech（F0無料枠）** の構成で、月額0円のまま安心して運用できます。
